

À LA CONQUÊTE DU CHEMIN OPTIMAL : PLONGÉE DANS LES ALGORITHMES HEURISTIQUES ET LA RIGUEUR IMPARABLE D'A*

presente par:
AZZMAN NOR ELIMANE
ELGHAZI SILIA



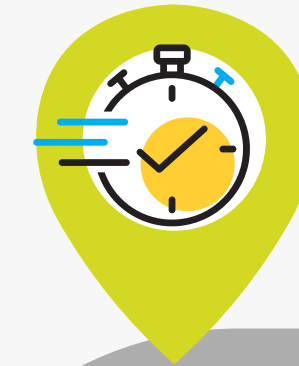
optimal path?



minimum cost?



higher
performance?



AlgorithmeA*

SOMMAIRE

01

Concepts Clés des
Algorithmes Heuristiques

03

L'Algorithme A en
profondeur*

05

Avantages et Limites de A*

02

Algorithmes Heuristiques
avant A*

04

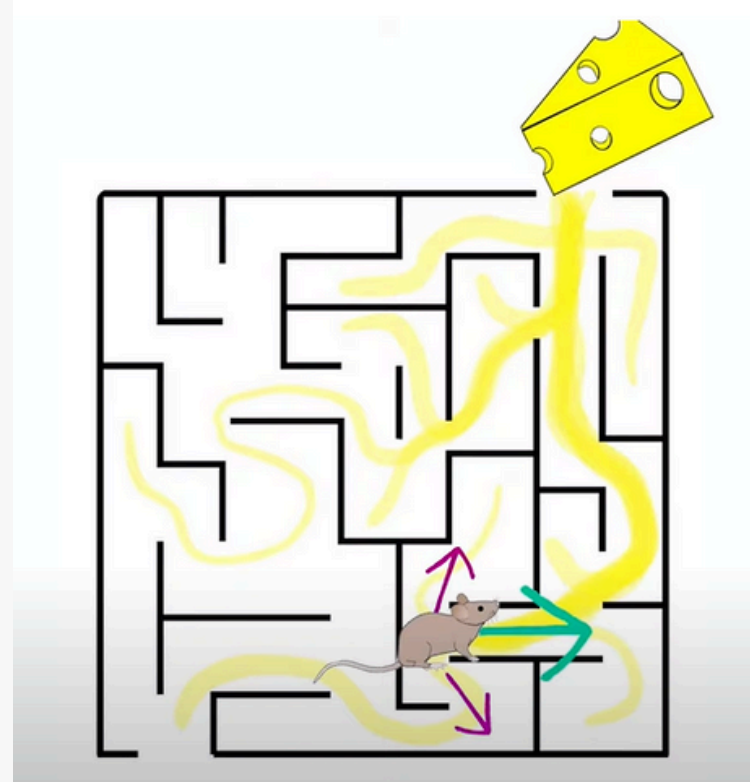
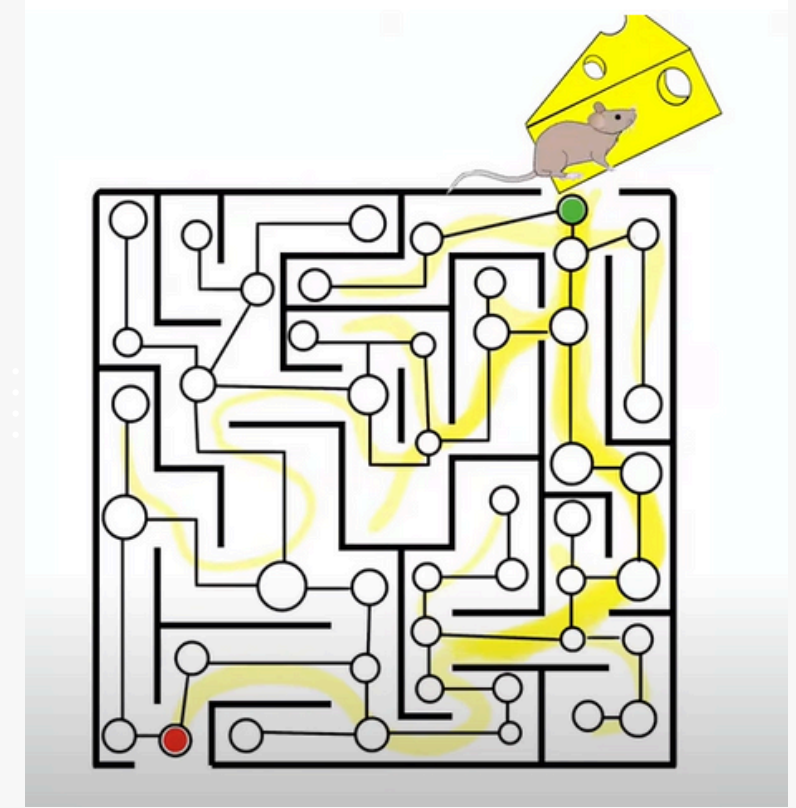
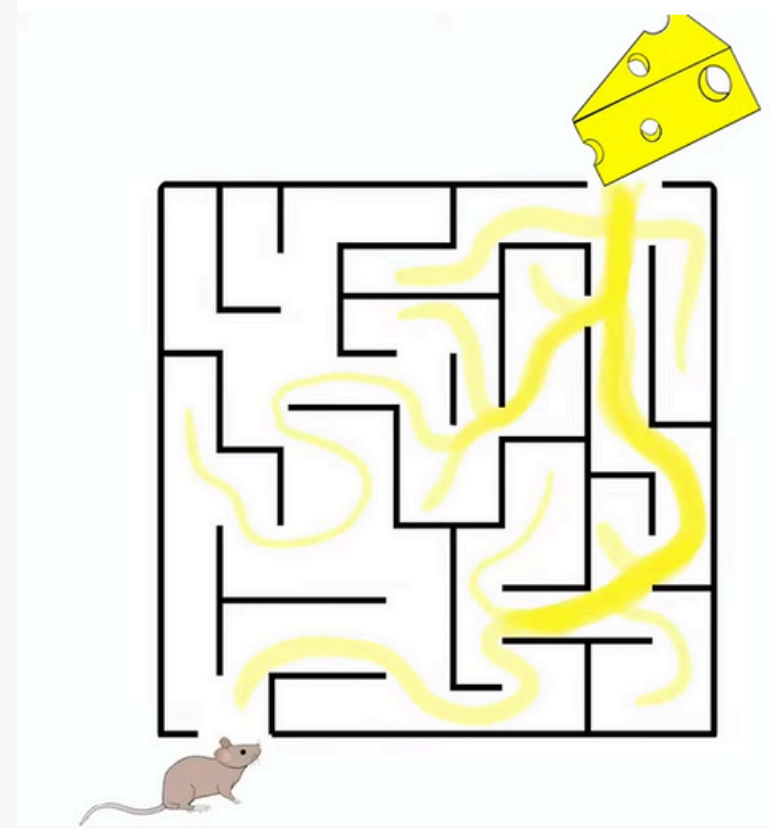
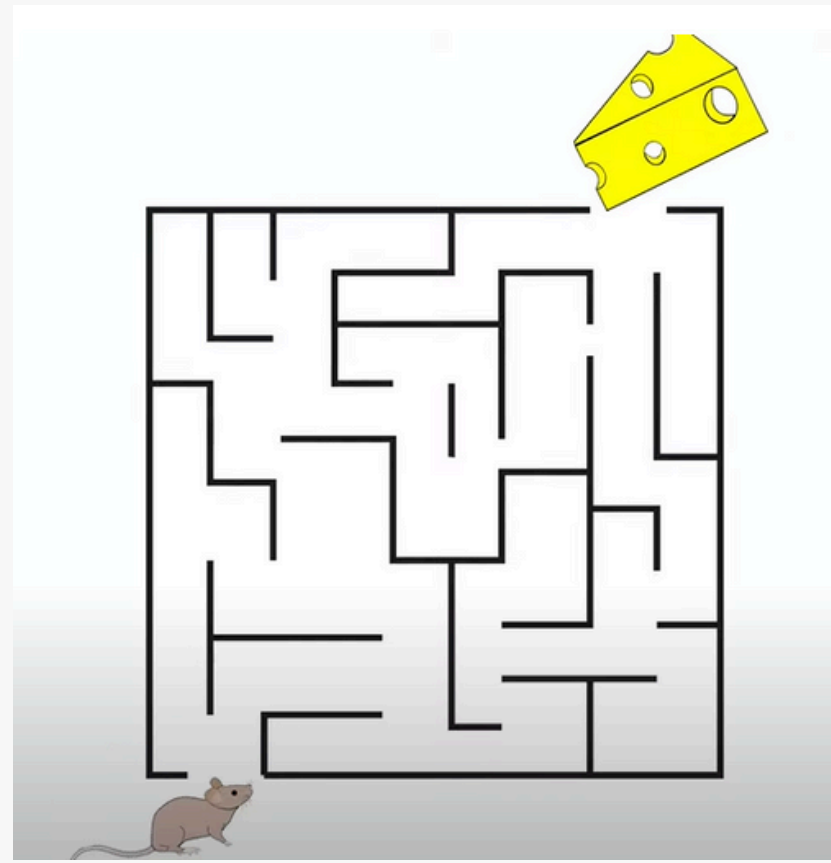
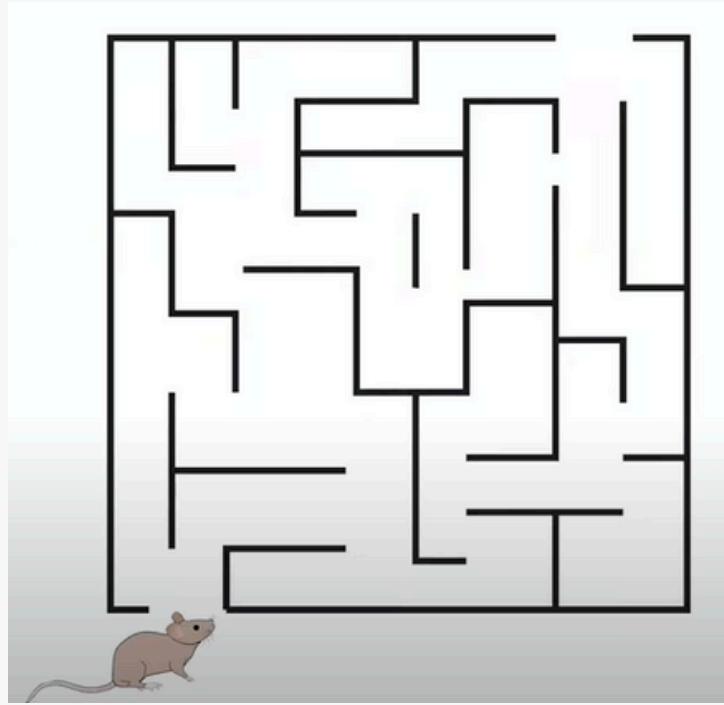
Implémentation pratique
détaillée de l'algorithme A*

06

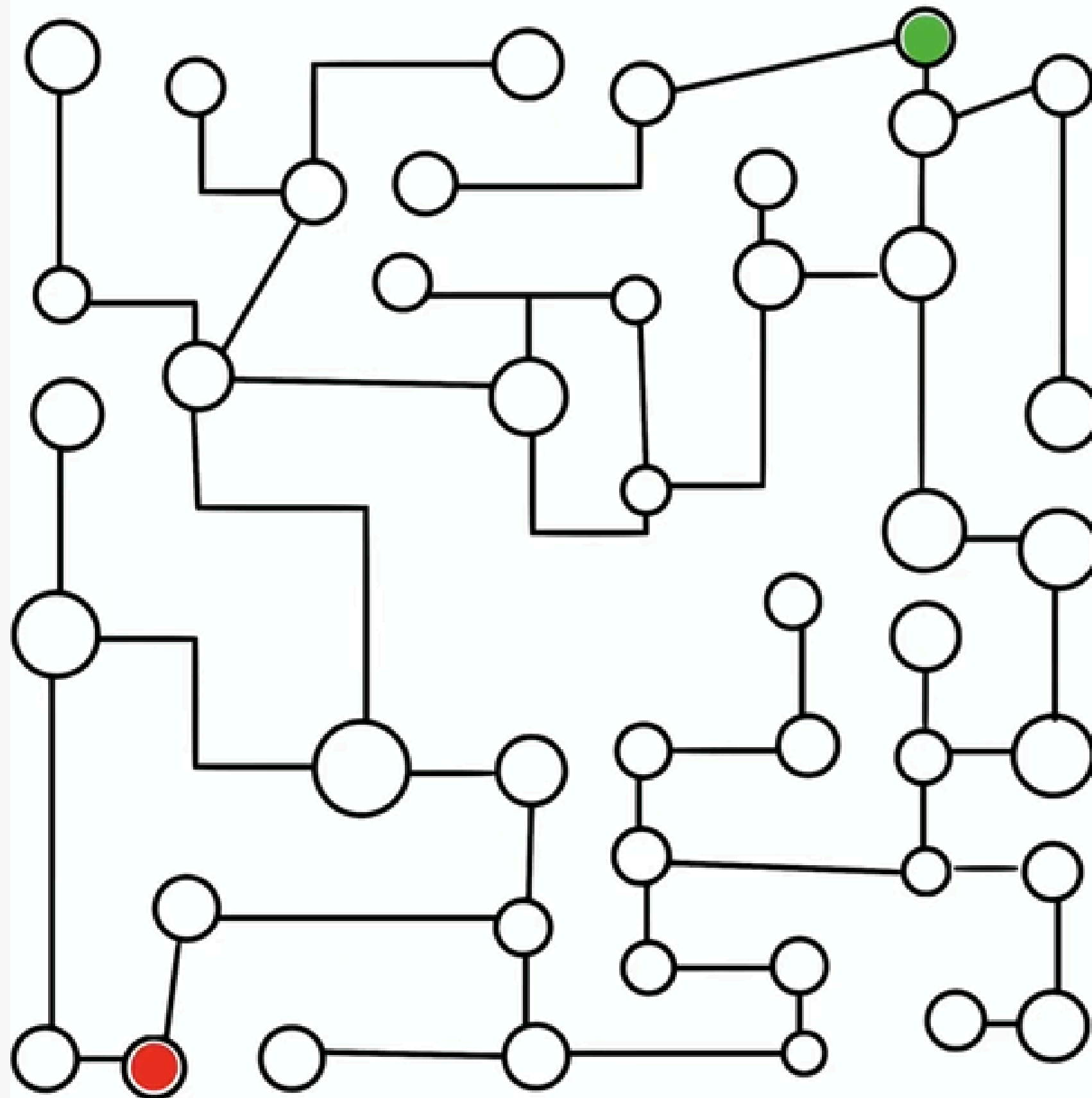
Domaines d'application réels

DÉFINITION D'UNE HEURISTIQUE





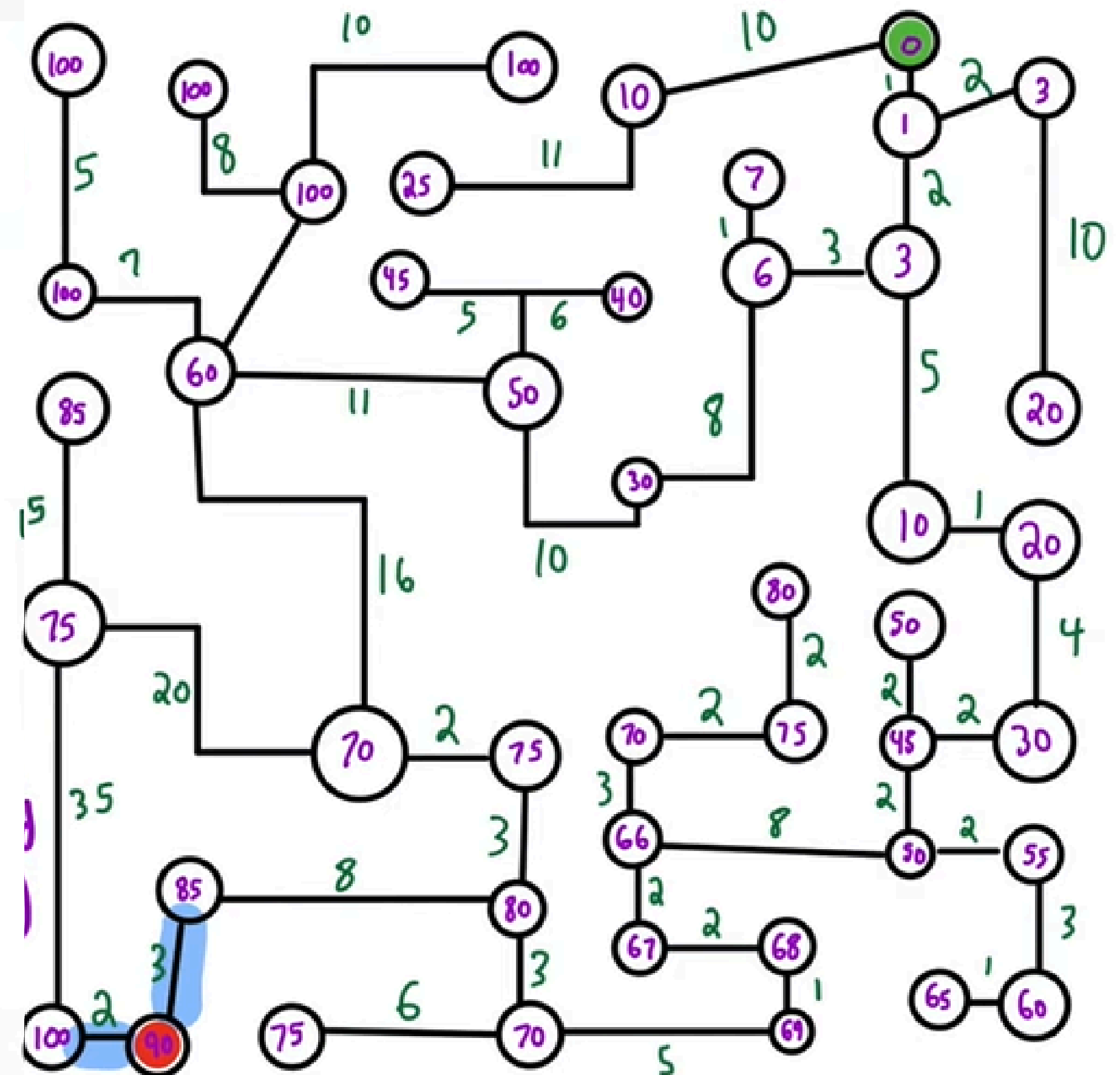
$$P(n) = g(n) + h(n)$$



90 : 0+90=90

85 : 3+85=88

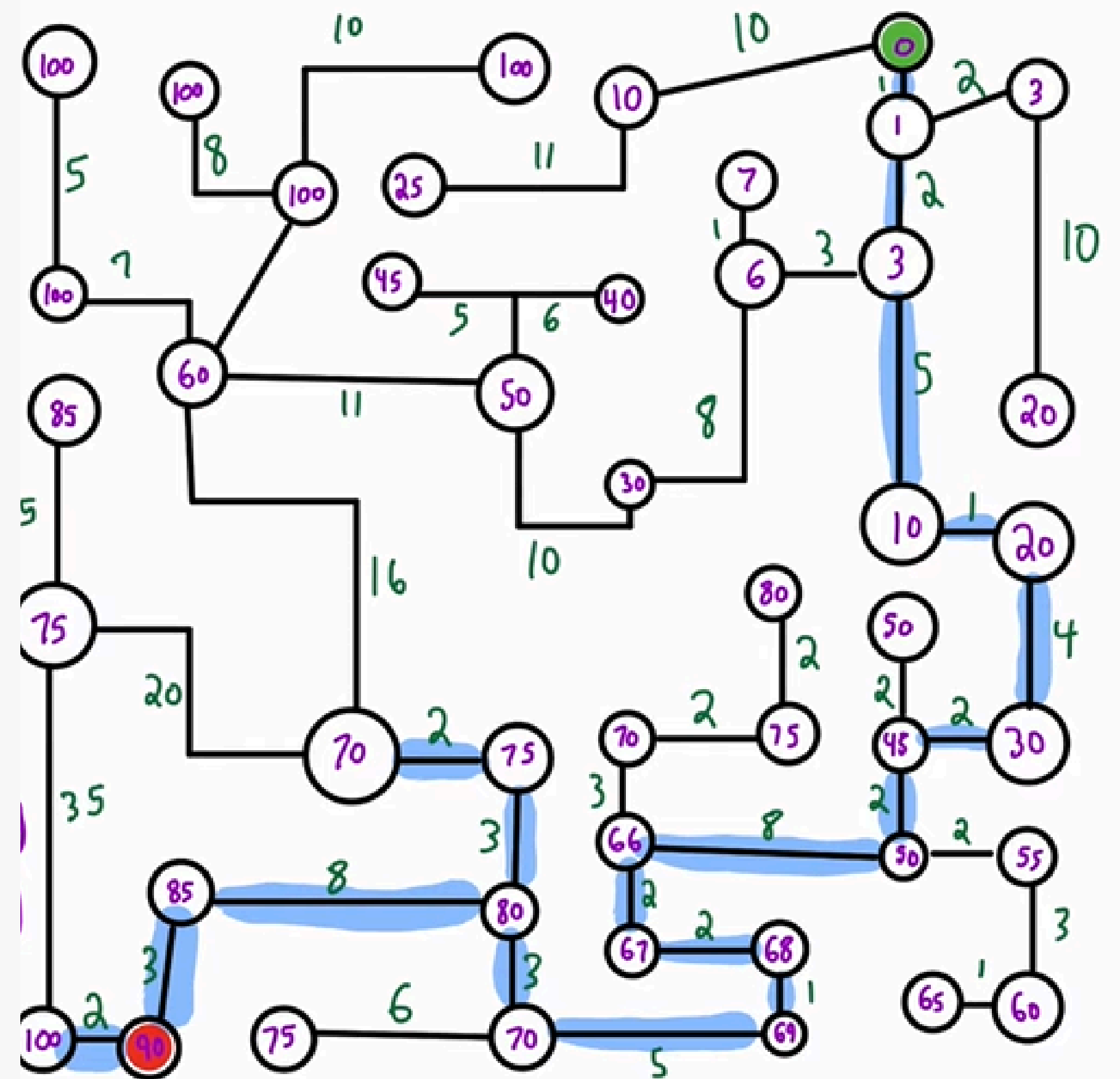
100 : 2+200=202



90 : 0+90=90

85 : 3+85=88

100 : 2+200=202



COMPARAISON AVEC AUTRES ALGOS

POURQUOI A* ?

Problème :

- Les heuristiques seules (comme la recherche gloutonne) vont vite mais peu fiables.
- Les algos comme Dijkstra sont fiables mais lents.



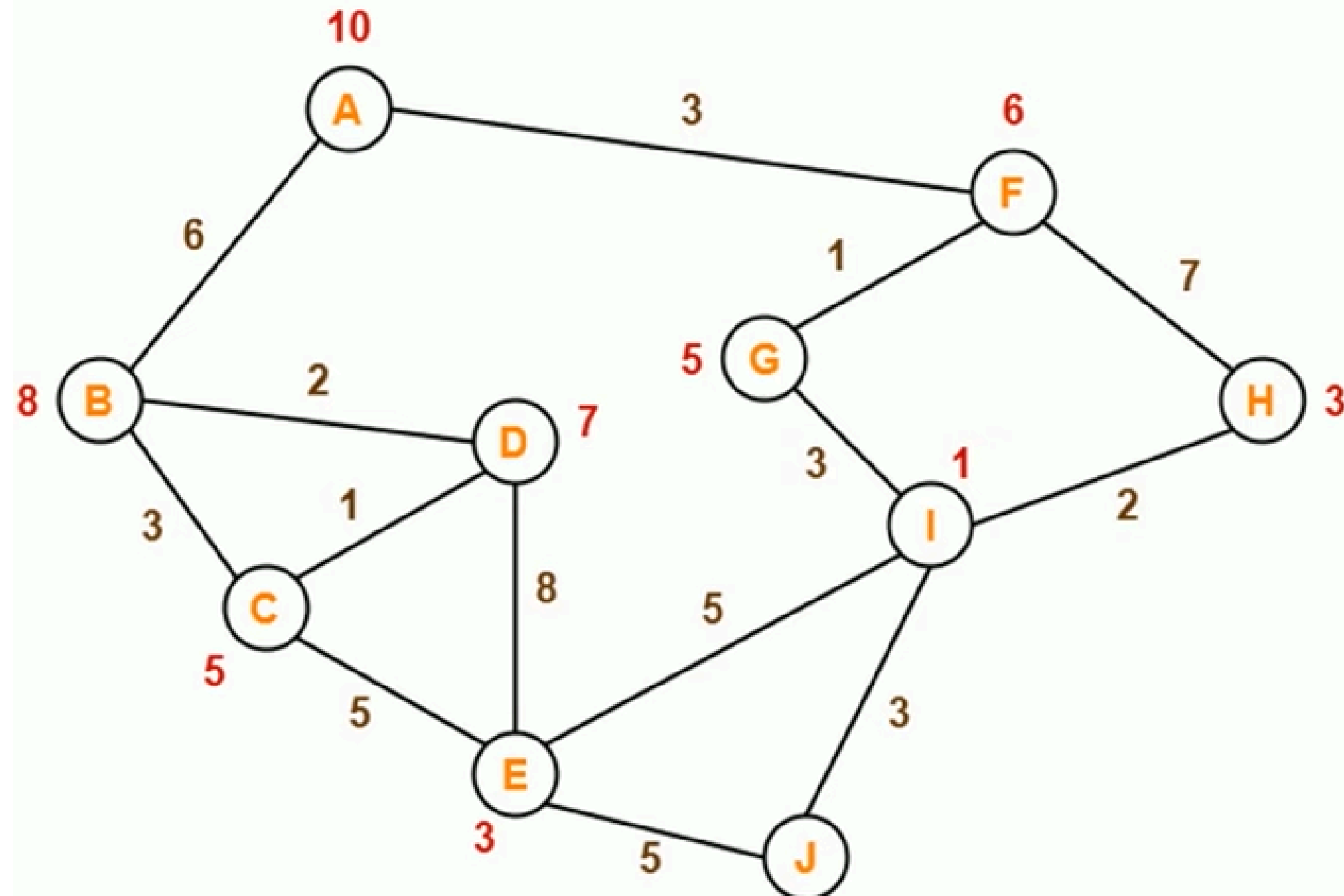
Solution :

- A* combine le meilleur des deux mondes :
- ✓ La rigueur de Dijkstra
- ✓ L'intuition de l'heuristique

DIJKSTRA VS HEURISTIQUE



ALGORITHME A*



Étape 0 – Initialisation :

On commence par le nœud A.

- Coût $g(A) = 0$ (point de départ)
- $h(A) = 10$ (estimation jusqu'à J)
- Donc $f(A) = 0 + 10 = 10$

Étape 1 : Exploration de A

Depuis A, on peut atteindre B et F

L'algorithme A* calcule $f(n) = g(n) + h(n)$

- B : $g = 6, h = 8 \rightarrow f(B) = 6 + 8 = 14$
- F : $g = 3, h = 6 \rightarrow f(F) = 3 + 6 = 9$

Comme $f(F) < f(B)$ donc on décide de choisir F.

Étape 2 : Exploration de F

Depuis F, on peut atteindre G et H.

Calcul de $f(n)$:

- G : $g = 3 + 1 = 4, h = 5 \rightarrow f(G) = 9$
- H : $g = 3 + 7 = 10, h = 3 \rightarrow f(H) = 13$

Comme $f(G) < f(H)$, on décide de choisir G.

ALGORITHME A*

Étape 3

Depuis G, on peut atteindre I.

Calcul de $f(n)$:

$$I : g = 4 + 3 = 7, h = 1 \rightarrow f(I) = 8$$

Il n'y a pas d'autre choix, on choisit I.

Étape 4

Depuis I, on peut atteindre H, E et J.

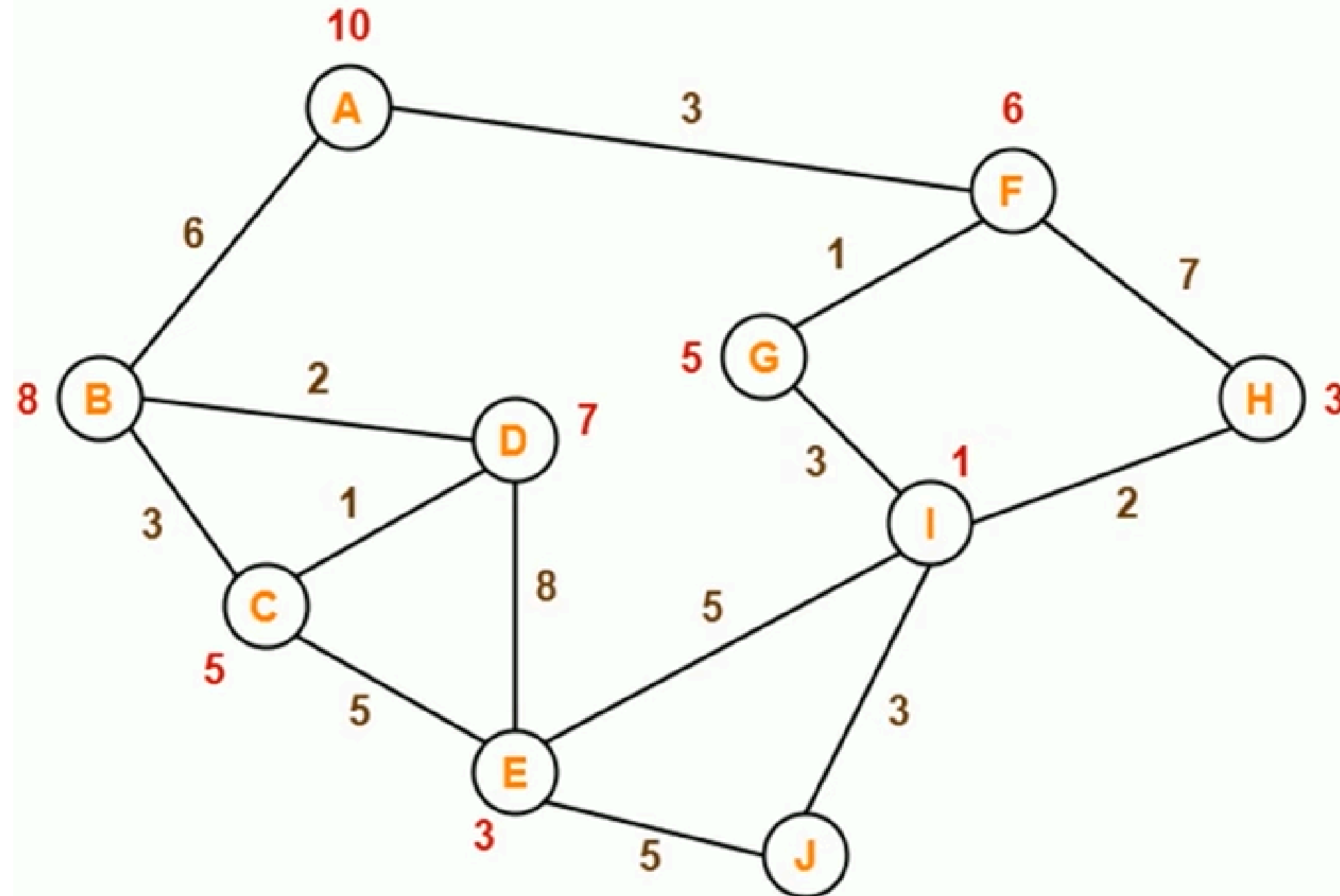
Calcul de $f(n)$:

$$H : g = 7 + 2 = 9, h = 3 \rightarrow f(H) = 12$$

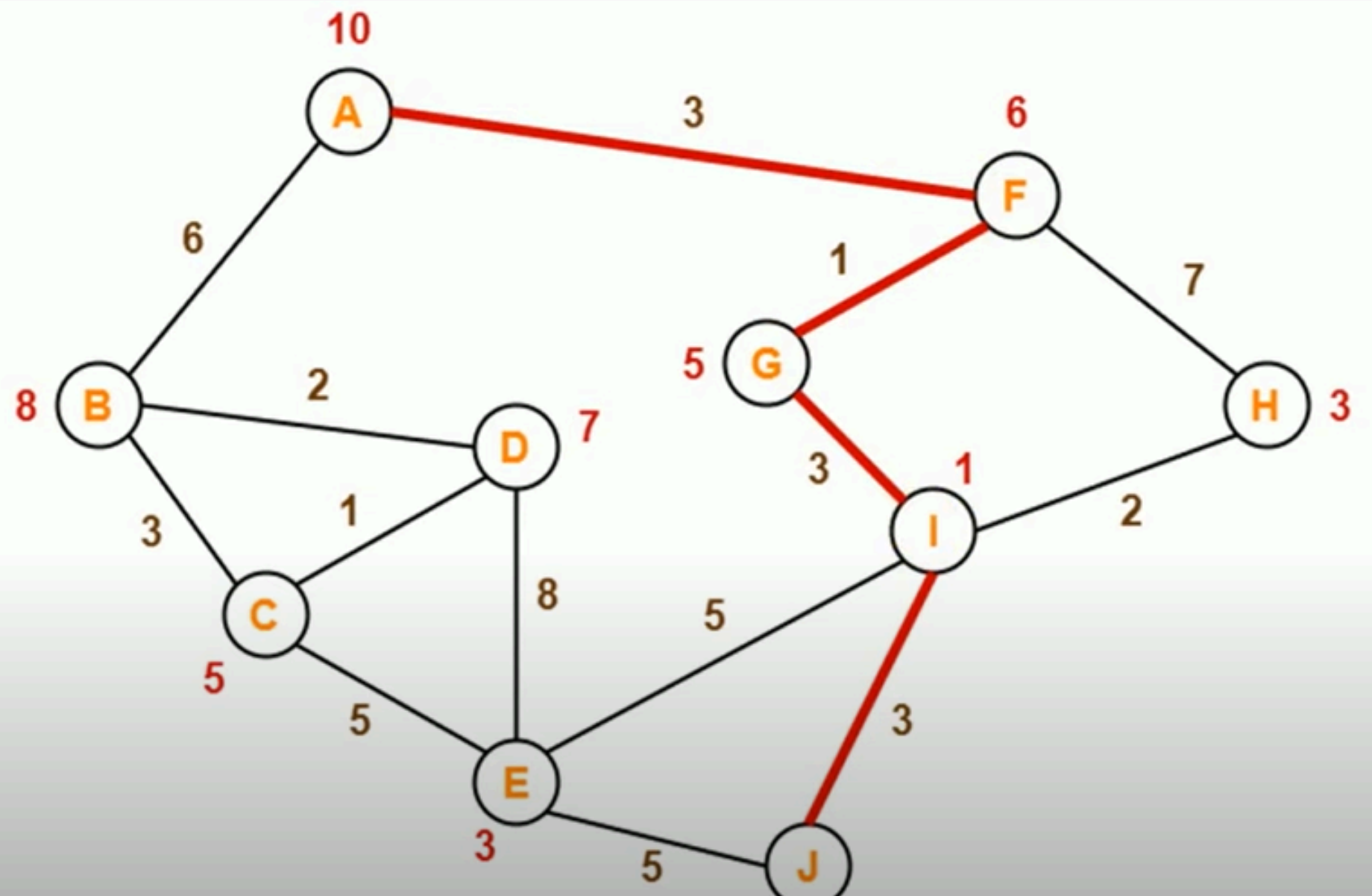
$$E : g = 7 + 5 = 12, h = 3 \rightarrow f(E) = 15$$

$$J : g = 7 + 3 = 10, h = 0 \rightarrow f(J) = 10$$

On choisit J car c'est le plus petit Car cest la destination.



ALGORITHME A*



$A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$

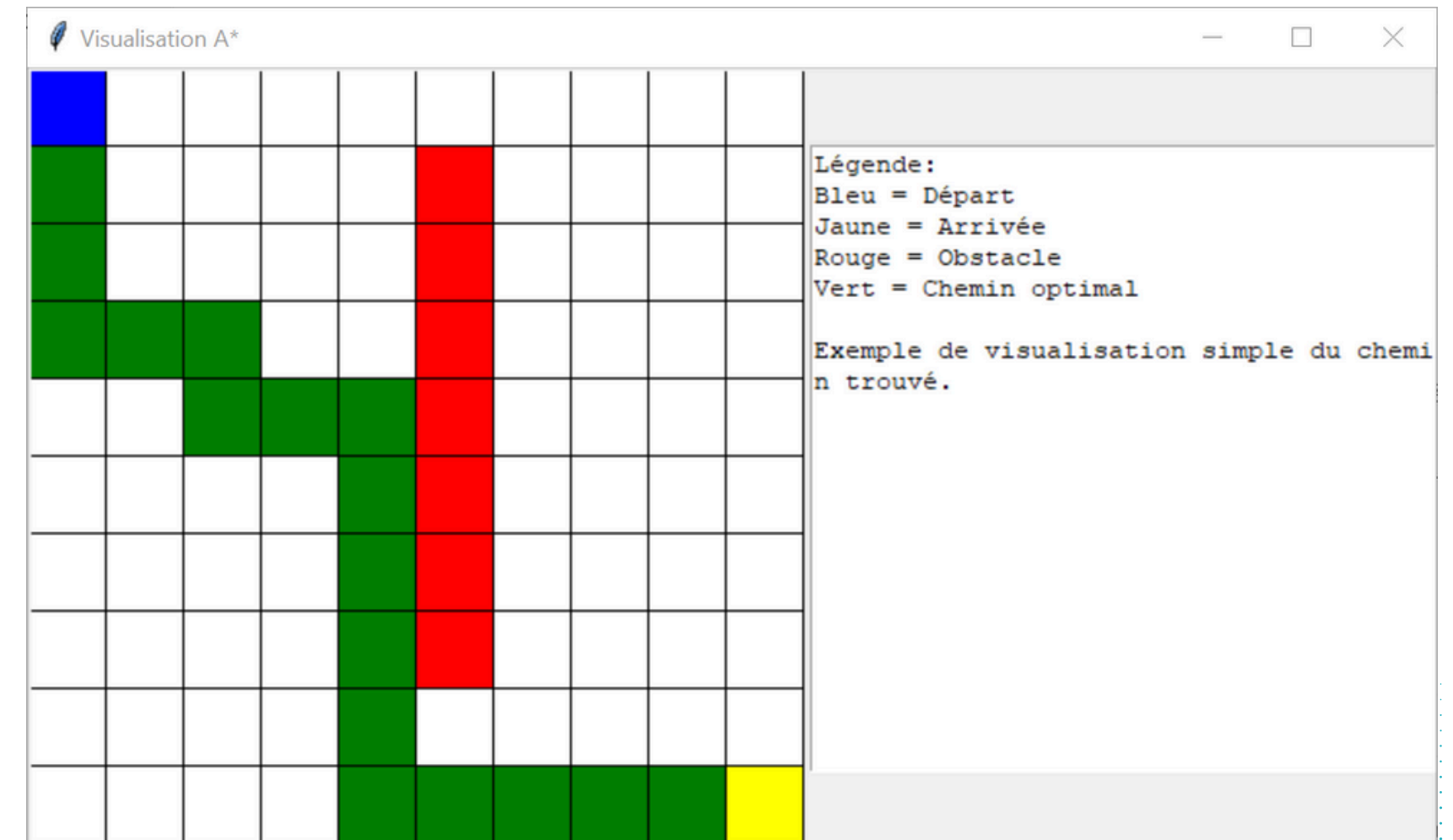
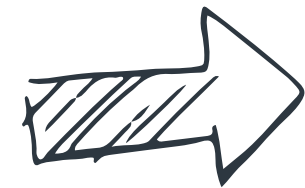
C'est le chemin optimal trouvé par A*

IMPLÉMENTATION PRATIQUE DÉTAILLÉE DE L'ALGORITHME A*

Ce projet implémente l'algorithme A* pour trouver le chemin optimal entre un point de départ et un point d'arrivée dans une grille 2D avec obstacles.

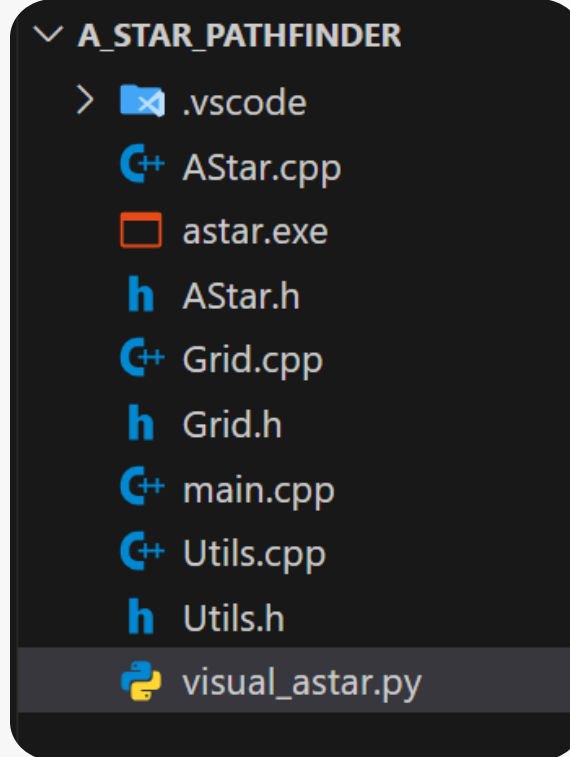
La grille est définie avec des cases libres ou bloquées, puis l'algorithme explore intelligemment les chemins possibles en utilisant une heuristique (distance de Manhattan) pour guider la recherche.

Le programme affiche dans la console la grille initiale, puis la grille avec le chemin optimal surligné, ainsi que les coordonnées du chemin trouvé.



INTERFACE GRAPHIQUE AVEC TKINTER

STRUCTURE DU PROJET ET RÔLES DES FICHIERS



Implémentation de l'algorithme A^* (logique de recherche du chemin)

Gestion de la grille 2D : obstacles, affichage console

Point d'entrée : configuration, lancement de l'algorithme, affichage final

Fonctions utilitaires (ex: reconstruction du chemin)

Version Python avec interface graphique Tkinter (visualisation intuitive)

AVANTAGES DE CETTE STRUCTURE

01 Modulaire et claire

02 Séparation logique

03 Possibilité d'avoir plusieurs interfaces

04 lisibilité

DISTANCE DE MANHATTAN

La distance de Manhattan entre deux points (x_1, y_1) et (x_2, y_2) dans une grille est la somme des distances absolues entre leurs coordonnées respectives :

$$\text{distance de Manhattan} = |x_1 - x_2| + |y_1 - y_2|$$

Pourquoi ce nom ?

- Imagine un plan de ville avec des rues formant un quadrillage (comme Manhattan à New York).
- Pour aller d'un point à un autre, on ne peut se déplacer qu'en suivant les rues (horizontalement ou verticalement), pas en ligne droite diagonale.
- La distance correspond donc au nombre de « blocs » qu'il faut parcourir.

Exemple concret

Si tu es à la case (2, 3) et que tu veux aller à la case (5, 7), la distance de Manhattan est :

$$|2 - 5| + |3 - 7| = 3 + 4 = 7$$

Tu dois donc faire 7 déplacements (3 vers la droite, 4 vers le haut par exemple).

AVANTAGES DE L'ALGORITHME A*

1

Optimalité garantie

A* garantit toujours de trouver le chemin le plus court entre deux points si l'heuristique utilisée ne surestime jamais la vraie distance.

2

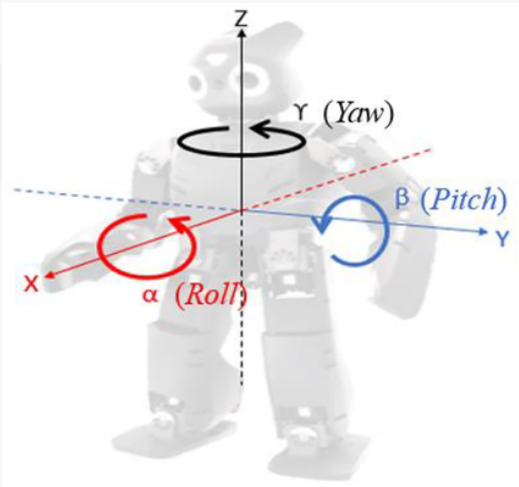
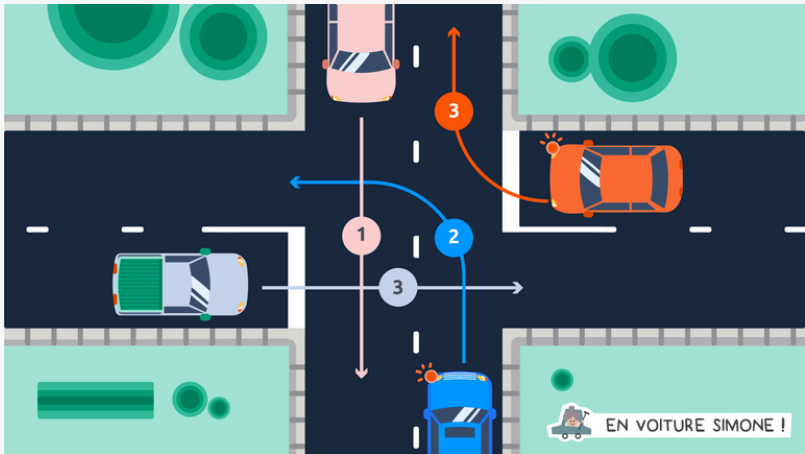
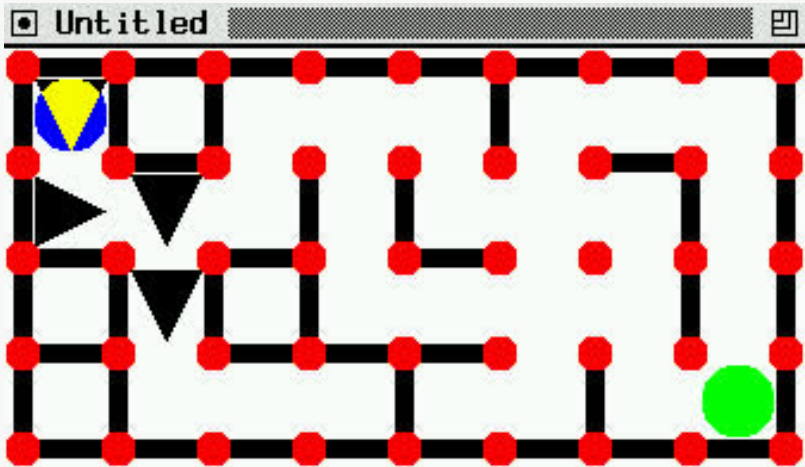
Efficacité

A* combine le coût déjà parcouru ($g(n)$) avec une estimation intelligente du coût restant ($h(n)$) pour explorer en priorité les chemins prometteurs. Cela évite d'explorer inutilement des chemins longs ou bloqués.

3

Flexibilité

L'algorithme peut être utilisé dans différents contextes en modifiant la fonction heuristique selon les contraintes spécifiques (par exemple, distance euclidienne, Manhattan, ou coûts variables).



LIMITES DE L'ALGORITHME A*

1

Consommation mémoire élevée

A* garde en mémoire tous les noeuds visités, ce qui peut devenir très gourmand en mémoire sur de grandes cartes ou graphes complexes.

2

Dépendance à l'heuristique

La performance et la précision dépendent fortement de la qualité de l'heuristique. Une heuristique mauvaise ou trop optimiste peut rendre l'algorithme lent ou non optimal.

DOMAINES D'APPLICATION MAJEURS DE L'ALGORITHME A*



Jeux vidéo

Pour gérer le déplacement intelligent des personnages et ennemis dans des environnements complexes (labyrinthes, cartes 3D), en trouvant rapidement des chemins optimaux.



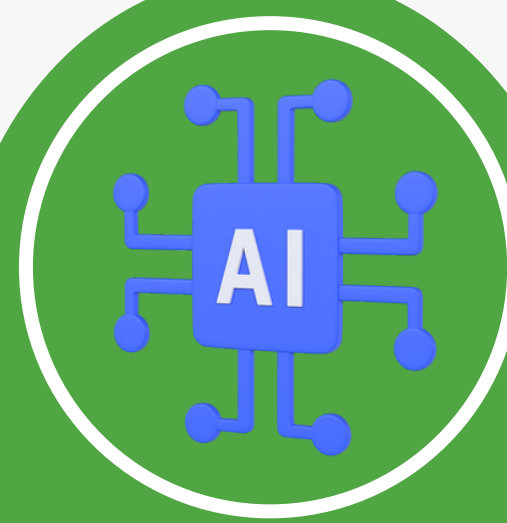
Robotique et véhicules autonomes

Pour la navigation en temps réel dans des espaces physiques avec obstacles, afin que les robots ou véhicules trouvent la route la plus sûre et efficace.



Systèmes de navigation et GPS

Pour calculer les itinéraires les plus courts ou rapides sur des réseaux routiers complexes en intégrant des données de trafic et distances.



Intelligence artificielle (IA) et recherche opérationnelle

Pour résoudre des problèmes d'optimisation et de recherche de chemin dans des graphes complexes, notamment dans les jeux d'échecs, puzzles, et simulations.



MERCI POUR VOTRE ATTENTION